BENI-SUEF UNIVERSITY · FACULTY OF ENGINEERING · COMM. & ELECTRONICS DEPT.

BitShield

Defending the Data Stream

Combined Error Detection & Correction with Incremental Redundancy (Type-II HARQ)

Supervisor: Dr. Mohamed Faysel

Academic Year 2025 – 2026

Team Members

1. Ahmed Toba Mahmoud
2. Ahmed Shaban Sayed
3. Adham Mahmoud Hamed
4. Mahmoud Saleh Awad
5. Mahmoud Ahmed
6. Hadeer Naser
7. Rana Tamer

github.com/ahmedtoba74/BitShield

Live Demo — Azure App Service

The Old System — Huffman + Hamming(7,4)

What we had before, and why it wasn't enough

✓ What It Detected & Corrected

- Single-bit errors per 7-bit block (corrected)
- Double-bit errors per block (detected only)
- Huffman compression reduced file size

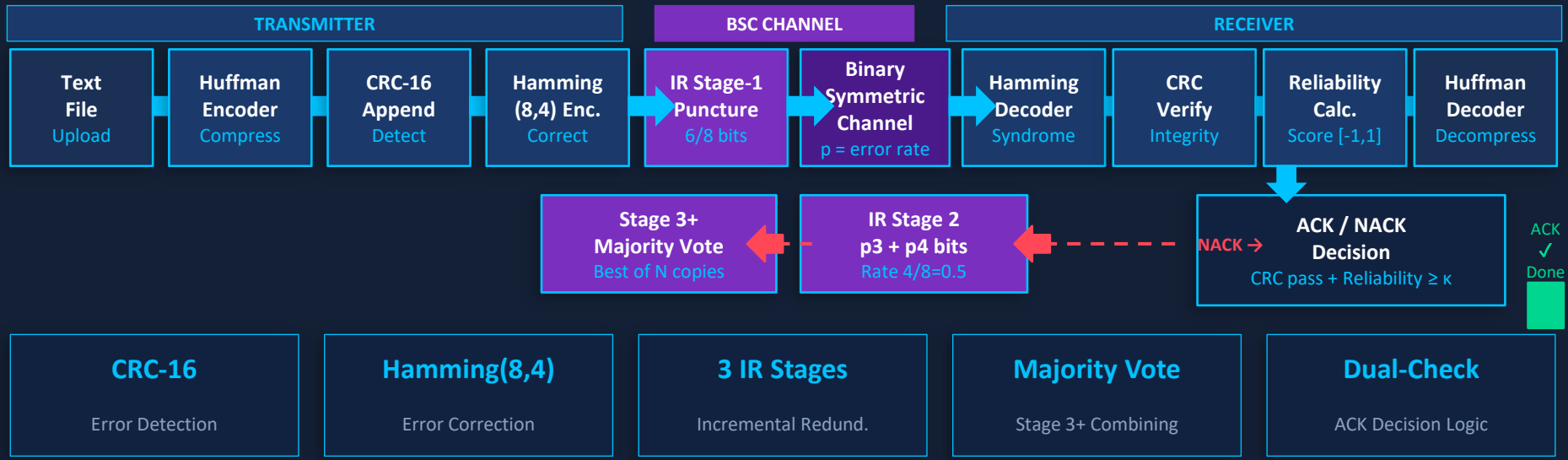
✕ Disadvantages & Gaps

- No CRC → silent mis-corrections undetectable
- Variable-length Huffman loses sync on 1 error
- No feedback: errors at 3+ bits cause silent failure
- Single-pass only — no retransmit mechanism
- Rate 4/7 wastes bandwidth at low noise

Why HARQ? → Combine ARQ feedback + FEC correction + incremental parity to fix ALL these gaps

New System — Full HARQ Pipeline

End-to-end flow: Transmitter → BSC Channel → Receiver → IR Loop



CRC-16	Hamming(8,4)	3 IR Stages	Majority Vote	Dual-Check
Error Detection	Error Correction	Incremental Redund.	Stage 3+ Combining	ACK Decision Logic

BitShield accepts a text file, compresses it with Huffman coding, protects it with CRC-16 and Extended Hamming(8,4), then transmits through a simulated Binary Symmetric Channel. At the receiver, a dual-criterion ACK/NACK loop decides whether to accept or request more parity — using only the incremental bits needed, not a full retransmit.

Software Specification

Technology stack, architecture & why each choice was made

Technology	Layer	Reason for Choice
Node.js v18+	Runtime	Non-blocking I/O; ideal for file-heavy simulation pipelines
Express.js v5	HTTP Framework	Minimal REST routing; async-native in v5
Multer v2	File Upload	Memory-buffer storage — no disk I/O for uploaded text files
Mulberry32 PRNG	Seeded Noise	Reproducible BSC — all 4 systems see identical bit flips
Chart.js (CDN)	Frontend Viz	Native canvas charts — BER/FER/Throughput rendered in browser
Font Awesome v6	UI Icons	CDN icons — zero build step
dotenv	Config	PORT and env management without hard-coding

Project Structure

```
app.js              → Express server entry point
src/services/       → 6 pure signal-processing modules (Huffman, CRC, Hamming, Noise, Reliability, IR)
src/controllers/    → HTTP request handlers (Classic + Simulation)
src/routes/api.js   → Route definitions (/api/process, /api/simulate, /montecarlo)
public/             → Single-page app: index.html + styles.css + app.js
```

Huffman Coding — Theory

Speaker 2 · Source Coding & Lossless Compression

Shannon Entropy

$$H(X) = -\sum P(x_i) \cdot \log_2 P(x_i)$$

Average code length L satisfies: $H(X) \le L < H(X) + 1$

Compression Ratio

$$CR = 1 - (\text{Compressed bits} / \text{Original bits})$$

Natural English text typically achieves CR ≈ 40 – 45%

Step 1	Step 2	Step 3	Step 4	Step 5
Frequency Analysis	**Build Priority Queue**	**Merge Lowest Two**	**Assign Codewords**	**Prefix-Free Property**
Count occurrences of each character → build probability map $P(x_i)$ = count / total.	Insert all symbols as leaf nodes; queue ordered by ascending frequency.	Repeatedly extract the two minimum-frequency nodes; create a parent node with frequency = sum of both; re-insert.	Traverse the tree: left edge = '0', right edge = '1'. Path from root to leaf = codeword.	No codeword is a prefix of another → unambiguous bit-stream decoding without delimiters.

Critical Risk: Variable-length codes have no synchronization boundaries. A single undetected bit error causes the decoder to lose alignment — corrupting ALL subsequent symbols. This is the primary motivation for CRC protection before the channel.

Huffman Coding — Implementation

huffmanService.js · 4 exported functions

Example Tree

```
Input: 'AABBC'
Freq: A=2, B=2, C=1


        [5]
       /   \
     [3]    [2]← 'C'=0
    /   \
 'A'=10  'B'=11

Codewords:
  A → 10    (freq=2)
  B → 11    (freq=2)
  C → 0     (freq=1)
```

Exported Functions — huffmanService.js

Function	Returns	Description
`analyzeProbabilities(text)`	{ frequencyMap, formattedOutput }	Counts characters, builds probability map
`buildHuffmanTree(freqMap)`	Node (tree root)	Priority-queue merge to build optimal prefix tree
`encode(text, freqMap)`	{ encodedBinary, huffmanTree, codesMap }	Converts text to compressed binary string
`decode(binaryString, root)`	string	Tree-traversal bit-by-bit decompression

Core Encode Logic

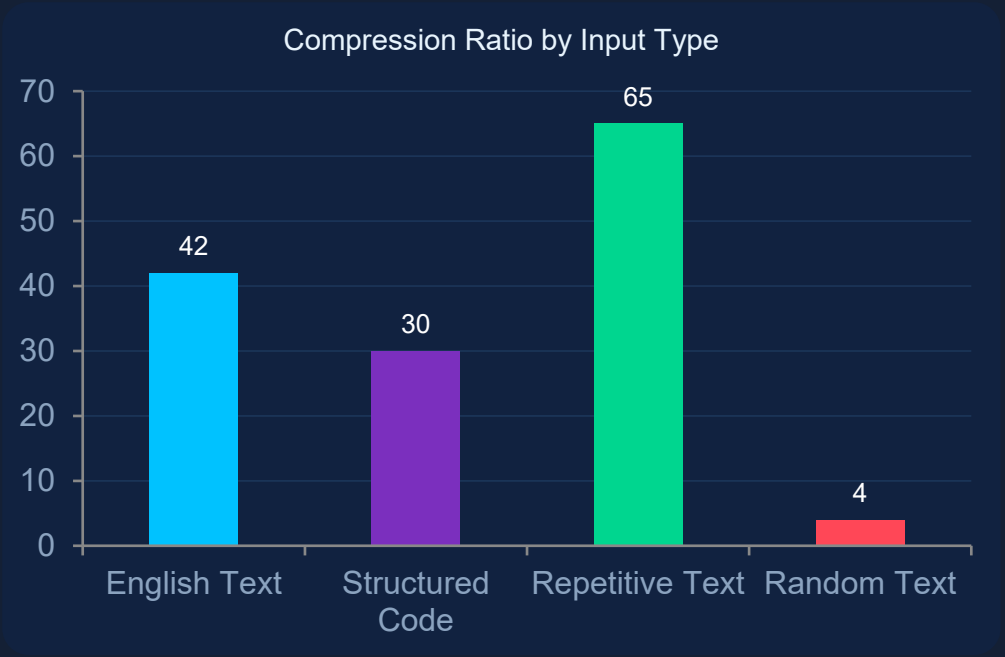
```
for (const char of text) {
  encodedBinary += codesMap[char];
}
// Prefix-free → unambiguous decode
// No separator bits needed
```

Key Properties

- Optimal: no other prefix-free code produces shorter average length
- Lossless: decode(encode(text)) === text always
- No sync markers needed — prefix-free property guarantees unambiguous decode
- Lossless: decode(encode(text)) === text always
- Optimal: no other prefix-free code produces shorter average length

Huffman Coding — Performance

Compression ratio analysis across input types



40–45%

Typical English compression

65%

Best case (repetitive text)

< 5%

Worst case (random/uniform)

Pipeline Position: Text File → [Huffman Encoder] → CRC-16 Append → Hamming(8,4) → BSC Channel

Huffman output feeds CRC before channel encoding — protecting the compressed representation

CRC-16-CCITT — Theory

Speaker 3 · Error Detection via Cyclic Redundancy Check

What is CRC?

CRC treats binary data as polynomial coefficients. The sender divides the data polynomial M(x) by a fixed generator G(x) using modulo-2 arithmetic, then appends the 16-bit remainder (checksum) to the frame. The receiver repeats the division — a non-zero remainder signals corruption.

CRC-16-CCITT Parameters

Polynomial: $x^{16} + x^{12} + x^5 + 1$

Hex: 0x1021 Width: 16 bits

Init: 0x0000 XOR-out: 0x0000

Error Pattern	CRC-16 Detects?	Detection Probability
All single-bit errors	✓ YES	100%
All double-bit errors	✓ YES	100%
All odd-number-of-bit errors	✓ YES	100%
Burst errors ≤ 16 bits	✓ YES	100%
Burst errors > 16 bits	~ YES	$1 - 2^{-16} \approx 99.998\%$
Undetectable random errors	✗ Possible	$P = 2^{-16} \approx 0.0015\%$

Pipeline Placement: [Huffman Output] → ⬛ CRC-16 Append ⬛ → Hamming(8,4) Encode → BSC

CRC is computed on compressed payload — protects the data the user cares about, before Hamming adds its own redundancy

CRC-16-CCITT — Implementation

crcService.js · 3 exported functions

Algorithm — Bitwise XOR

```
Generator:  0x1021 = 0001000000100001

1. Initialize CRC register = 0x0000
2. For each bit in data:
   a. XOR top bit of CRC with data bit
   b. Shift CRC left by 1
   c. If XOR result was 1:
      CRC = CRC XOR 0x1021
3. Append 16-bit CRC to payload
4. At receiver: re-run division
   remainder = 0x0000 → frame OK
   remainder ≠ 0x0000 → error detected
```

Exported Functions — crcService.js

Function	Returns	What it does
`computeCRC(binaryString)`	16-bit CRC string	Bitwise shift-register division over input bits
`appendCRC(binaryString)`	{ protected, payloadLength }	Appends CRC to data; records payload length for later extraction
`checkCRC(data, payloadLen)`	{ isValid, payload }	Re-runs CRC over received data; separates payload from CRC field

Integration in Pipeline

TX: Huffman bits → appendCRC() → Hamming encode
RX:
Hamming decode → checkCRC() → isValid?
CRC error on RX = trigger NACK in IR loop

Why CRC-16-CCITT specifically? The 0x1021 polynomial was chosen by ITU-T because it is particularly good at detecting burst errors in framed serial data — exactly the scenario of a noisy digital channel.

CRC — Worked Example & Role in HARQ

How CRC catches what Hamming misses

Example: 3-Bit Error Scenario

```
Payload (4 bits):          1011
After Hamming(8,4):        10110110   (8 bits)
3 bits corrupted by BSC:   10011110   ← errors

Hamming syndrome (3+err): may point to wrong position
→ Hamming 'corrects' to wrong value (silent failure)

CRC check on decoded bits: FAIL   (remainder ≠ 0)
→ CRC catches the mis-correction!
→ IR loop issues NACK → requests more parity
```

Why the Combination Matters

Hamming alone: silent failure at 3+ errors

CRC alone: detects but cannot correct

Together: CRC catches Hamming's misses → NACK → IR sends more parity

Result: no silent failures at any noise level

2^{-16}

Undetected error probability
≈ 0.0015%

100%

Detection of all
≤ 16-bit bursts

16 bits

Overhead added
to every frame

CRC is the safety net that makes HARQ trustworthy — it is what allows the system to issue a confident ACK.

Extended Hamming(8,4) — Theory

Speaker 4 · Forward Error Correction

Property	Hamming(7,4)	Extended Hamming(8,4) ✓
Data bits k	4	4
Total bits n	7	8
Code rate R	4/7 ≈ 0.571	4/8 = 0.500
Min. distance d_min	3	4 ★

8-bit Codeword Layout

Parity Equations

$$p1 = d1 \oplus d2 \oplus d4$$

$$p2 = d1 \oplus d3 \oplus d4$$

$$p3 = d2 \oplus d3 \oplus d4$$

$$p4 = d1 \oplus d2 \oplus d3 \oplus d4 \oplus p1 \oplus p2 \oplus p3$$

Syndrome (s1,s2,s3)	Overall Parity p4	Interpretation	Action
000	OK (=0)	No error	Pass to CRC
≠ 000	FAIL (=1)	Single-bit error	Flip bit at syndrome position
≠ 000	OK (=0)	Double-bit error	Issue NACK → IR
000	FAIL (=1)	p4 itself flipped	Data safe — ignore

d_min = 4 → t = 1 error correctable | Detection of all 2-bit errors (without correction)

Extended Hamming(8,4) — Implementation

hammingService.js · 5 exported function groups

`encode(binary)`	Legacy Hamming(7,4) for Classic Mode.
`decode(noisy)`	Syndrome-based 7,4 decode with correction report.
`encodeExtended(binary)`	Extended(8,4) — adds p4 overall parity. Returns paddingBits for alignment.
`decodeExtended(enc, pad)`	Syndrome decode with 2-error detection. Returns per-block report used by reliability service.
`encodePunctured(binary)`	Splits each 8-bit codeword → Stage1 bits (d1-d4,p1,p2) and Stage2 bits (p3,p4) for IR.
`decodeFromStages(s1,s2,pad)`	Reconstructs full codeword from IR stages; inserts zeros for missing parity positions.
`majorityVoteCombine(copies[])`	Accepts array of received copies; returns bit-wise majority vote for Stage 3+ combining.

Legacy 7,4 · Extended 8,4 · IR Stage Split · Majority Combining

Hamming — IR Puncturing & Majority Vote

How the 8-bit codeword is split across IR stages

Stage 1 (Punctured)	Stage 2 (Full Code)	Stage 3+ (Combining)
Rate 4/6 ≈ 0.67	**Rate 4/8 = 0.50**	**Effective R ↓**
Transmitted Bits:	Transmitted Bits:	Transmitted Bits:
`d1 d2 d3 d4 p1 p2`	`p3 p4 (only!)`	`Full 8-bit retransmit`
p3 p4 withheld	Full decode now possible	Majority vote on all copies

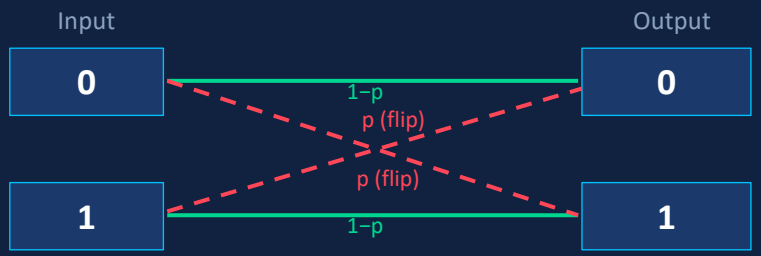
Majority Vote Combining — Stage 3+ Example

```
Received copy 1:  1 0 1 1 0 1 1 1   (1 bit flipped by BSC)
Received copy 2:  1 0 0 1 0 1 1 0   (1 bit flipped by BSC)
Received copy 3:  1 0 1 1 0 1 1 0   (no error)
Majority vote:    1 0 1 1 0 1 1 0   ✓ Correct original
```

Binary Symmetric Channel — Theory

Speaker 5 · Channel Model & Noise Simulation

BSC Model




```
P(rcv=0 | sent=0) = 1−p     P(rcv=1 | sent=0) = p
P(rcv=1 | sent=1) = 1−p     P(rcv=0 | sent=1) = p
```

Mulberry32 — Seeded PRNG

BitShield uses Mulberry32, a fast 32-bit seeded pseudo-random number generator, rather than Math.random().

Why seeded PRNG?

- All 4 comparison systems receive bit-for-bit identical noise sequences
- Performance differences → protection strategy only (not luck)
- Monte Carlo trials are fully reproducible
- Fair academic comparison with zero confounding factors

Error Rate p	P(block clean)	P(1-bit error)	P(2-bit error)
0.001 (0.1%)	99.21%	0.78%	0.003%
0.01 (1%)	92.27%	7.46%	0.26%
0.05 (5%)	66.34%	27.94%	5.19%
0.10 (10%)	43.05%	38.26%	14.88%

BSC Channel — Implementation

noiseService.js · Random & Seeded Noise Injection

injectNoise(binary, rate)

Classic Mode (non-reproducible)

```
for (let i = 0; i < bits.length; i++) {
  if (Math.random() < rate) {
    bits[i] = bits[i] === '0' ? '1' : '0';
    flippedPositions.push(i);
  }
}
return { noisyData, noiseReport };
```

Returns: noiseReport = { flippedCount, flippedPositions, totalBits }

Used in: Classic Mode (/api/process)

injectNoiseSeeded(binary, rate, seed)

HARQ / Compare / Monte Carlo Mode

```
// Mulberry32 PRNG
let s = seed + 0x6D2B79F5;
function rand() {
  s = Math.imul(s ^ s>>>15, s|1);
  s ^= s + Math.imul(s^s>>>7, s|61);
  return ((s ^ s>>>14) >>> 0) / 4294967296;
}
```

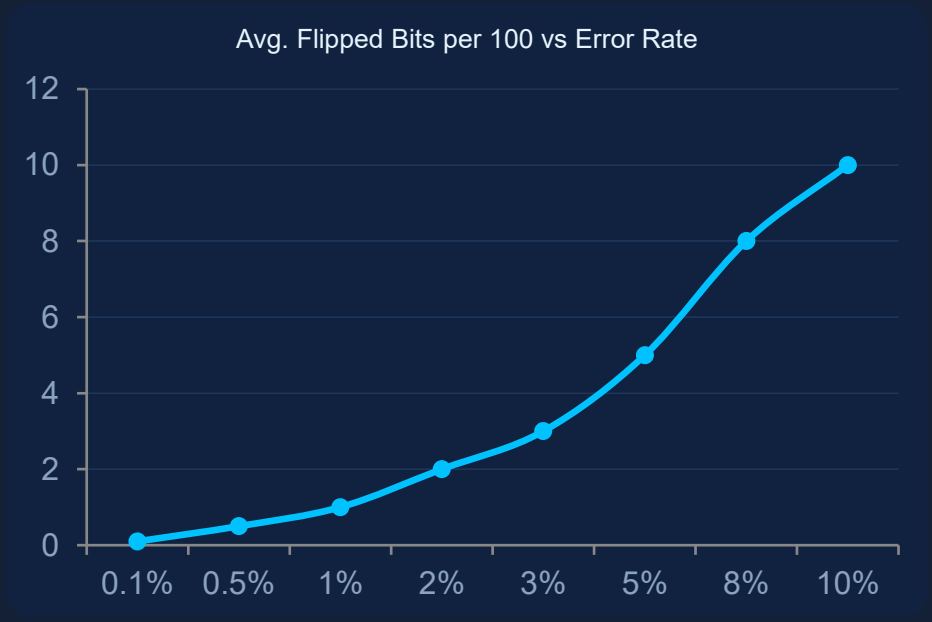
All 4 systems pass the same seed → identical bit flips

Used in: /api/simulate, /compare, /montecarlo

Monte Carlo sweep runs up to 200 trials per noise point × 15 points = 3,000 independent BSC simulations. Each trial uses a unique seed derived from the trial index, ensuring statistical independence while remaining reproducible.

BSC — Noise Report & Channel Behavior

How errors distribute across the bit stream



BSC Behavioral Characteristics

- BSC flips each bit independently — no memory, no burst correlation
- P(flip at position i) = p, regardless of adjacent bits
- At p = 0.01: ~7.5% of 8-bit blocks have exactly 1 error (correctable by Hamming)
- At p = 0.05: ~28% of blocks have 1 error, ~5% have 2 errors (detected but uncorrectable)
- At p = 0.10: ~38% have 1 error, ~15% have 2 errors — Hamming is near saturation
- Seeded PRNG ensures all 4 comparison systems face identical flips per trial

Memoryless

Each bit flip is independent
— no burst correlation

Symmetric

P(0→1) = P(1→0) = p
Exact same crossover

Configurable

p range: 0.001 – 0.10
for HARQ simulation

Reliability Score & IR ACK/NACK — Theory

Speaker 6 · Dual-Criterion Decision Logic

Reliability Score Formula

```
reliability = (cleanBlocks − 2 × uncorrectableBlocks)
                        / totalBlocks
```

Range: [−1, +1] +1 = perfect −1 = all blocks uncorrectable

Inspired by Chapter 17 syndrome-weight soft metric

Adaptive Threshold κ

```
threshold = 1 − κ × P(blockErr) × d_min
```

κ = user-configurable multiplier (default 1.0)
d_min = 4 (Extended Hamming(8,4))

Dual-Criterion ACK Decision

BOTH conditions must be TRUE → ACK

Condition 1: CRC check PASS

Payload polynomial division remainder = 0

Condition 2: Reliability ≥ Threshold

Syndrome-weight score exceeds adaptive κ threshold

CRC alone can pass by chance

$P = 2^{-16} \approx 0.0015\%$ — rare but real. Reliability adds a second independent check.

Reliability alone is soft

High reliability doesn't guarantee zero errors. CRC provides algebraic certainty on the payload.

Together = strong guarantee

Both passing simultaneously makes false ACK essentially impossible under operational error rates.

Reliability & IR Controller — Implementation

reliabilityService.js + irService.js

reliabilityService.js

`computeReliability(report)`

Reads per-block syndrome report. Scores +1 for clean, 0 for corrected single-error, −2 for uncorrectable double-error blocks. Normalizes to [−1, +1].

`computeThreshold(κ, errorRate)`

Calculates 1 − κ × P(blockErr) × d_min. Provides adaptive ACK stringency — higher κ = stricter.

`makeDecision(crcPass, rel, thresh)`

Returns { decision: 'ACK'|'NACK', reason }. Both CRC and reliability must pass for ACK.

irService.js — IR Controller

`createIRSession(text, opts)`

Orchestrates the full HARQ pipeline. Loops through IR stages on NACK. Records per-stage metrics: transmitted bits, errors, correction counts, decision.

`runComparativeSimulation(text, opts)`

Creates 4 independent pipelines sharing the same PRNG seed. Returns side-by-side metrics for all systems.

`runMonteCarlo(text, opts)`

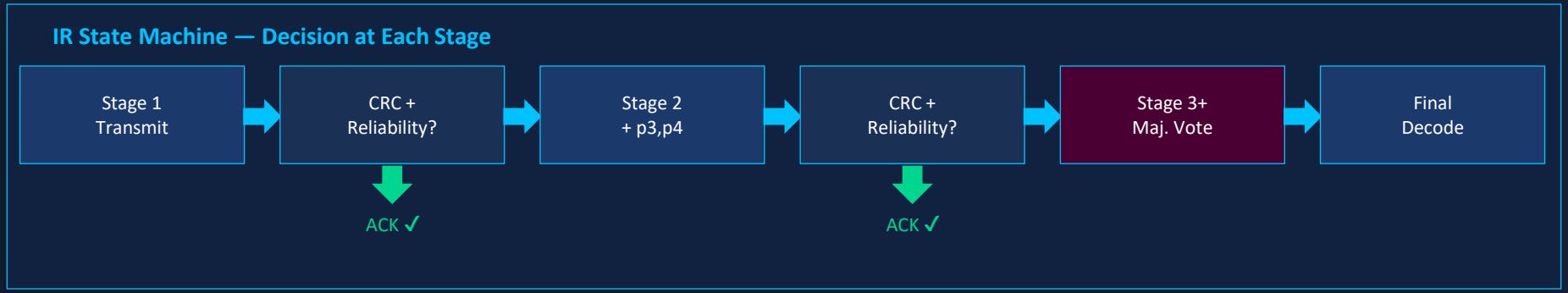
Sweeps errorRate from min to max across numPoints. Runs trialsPerPoint trials at each point. Returns BER[], FER[], throughput[] arrays ready for Chart.js.

IR Loop Logic

IR Stages — ACK/NACK Decision Detail

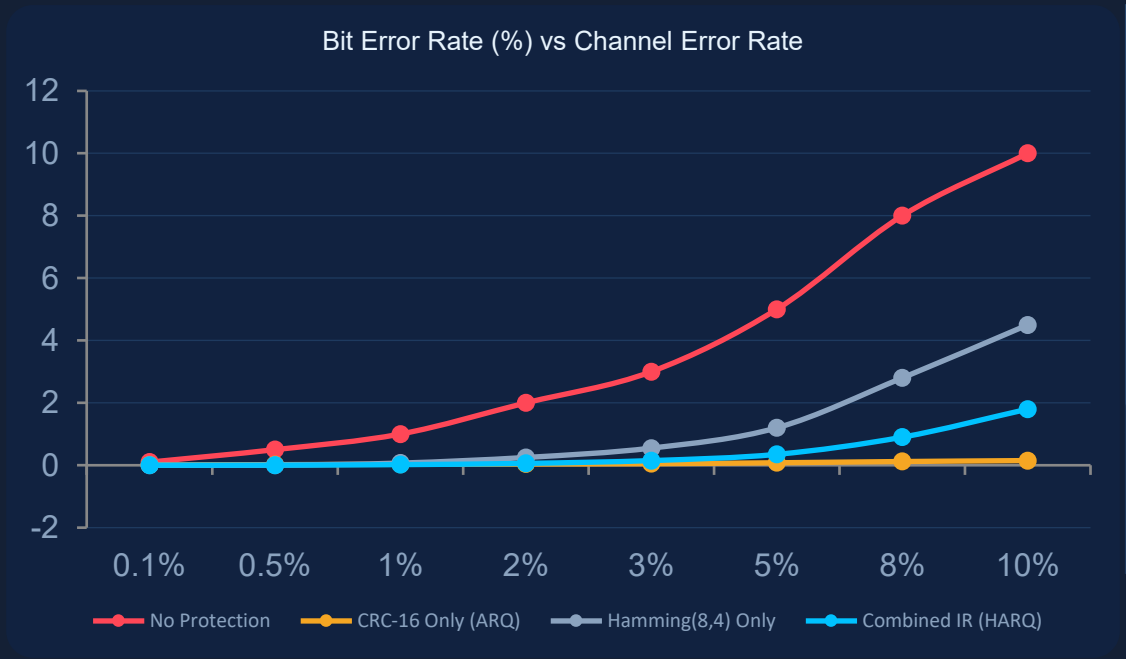
How the system adapts redundancy to channel conditions

κ Value	Threshold Behavior	Effect	Trade-off
κ = 0.5	Loose threshold	Accepts more frames at Stage 1	Higher throughput, slightly higher BER
κ = 1.0 (default)	Balanced threshold	Standard ACK criterion	Best BER-throughput balance
κ = 2.0	Strict threshold	Demands high confidence before ACK	Lower BER, more retransmissions

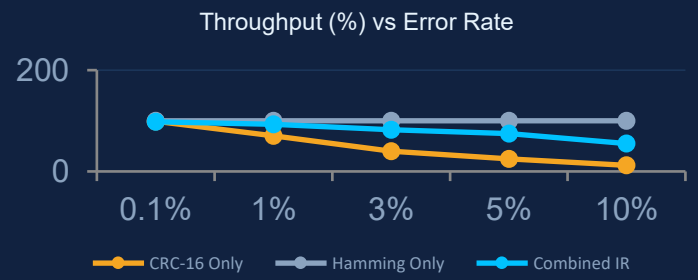


Results — 4-System Performance Comparison

Speaker 7 · Monte Carlo BER / FER / Throughput



System	BER @ 1%	FER @ 1%	Throughput	Silent Failure?
No Protection	~1%	~8%	100% (nominal)	YES ✗
CRC-16 Only (ARQ)	~0%	~0% after ARQ	~70% (retransmits)	NO ✓
Hamming Only	~0.07%	~3%	100% (FEC only)	YES ✗



Live Demo — System in Action

Test case walkthrough on Azure-deployed web application

Live URL: signalprocessing-fnexbfhvf9bwc4hh.polandcentral-01.azurewebsites.net

Tab 1: Classic Mode

Upload .txt file → Huffman + Hamming(7,4) single-pass → view compression ratio, errors injected/corrected, recovered text vs original

Tab 2: HARQ Simulation

Configure error rate, max stages, κ → run IR loop → view per-stage timeline showing transmitted bits, corrections, ACK/NACK decision at each stage

Tab 3: Performance Analysis

Set Monte Carlo parameters → run sweep → view 4 interactive Chart.js plots: BER, FER, Throughput, Avg Retransmissions — all 4 systems overlaid

Test Case Walkthrough — HARQ Simulation at p = 0.03

Input	Upload test_Project.txt (~1.6 KB natural English text)
Compression	Huffman: 13,824 bits → 7,892 bits | CR = 42.9%
CRC Append	7,892 bits + 16-bit CRC → 7,908 bits
Hamming(8,4)	7,908 bits → 15,816 bits (doubly padded to 4-bit blocks)
IR Stage 1	Transmit 11,862 bits (6/8 per block) | BSC flips ~355 bits (p=0.03)
Decision	CRC FAIL → NACK → Request Stage 2
IR Stage 2	Transmit remaining 3,954 parity bits | Full decode: corrects single-bit errors
Decision	CRC PASS + Reliability ≥ κ → ACK ✓ | Recovered text matches original

Conclusion

- Combined IR (HARQ) achieves the best BER and FER at all tested noise levels without sacrificing throughput to the extent of pure ARQ.
- CRC-16 catches silent mis-corrections that Hamming alone cannot detect — making it an essential layer on top of FEC.
- Incremental Redundancy reduces retransmission cost by sending only withheld parity bits, not full frame repeats.
- The dual-criterion ACK (CRC + reliability score) eliminates false ACKs that CRC alone might occasionally pass.
- Seeded PRNG ensures fair, reproducible comparison — all systems face identical noise per trial.

Future Work

- Replace BSC with Gilbert-Elliott burst-error channel model + convolutional interleaving
- Implement soft-decision LLR combining (operational LTE/5G standard) for 3–5 dB gain
- Extend to LDPC or Polar codes — superior performance at longer block lengths
- Real-time streaming simulation with frame-by-frame visualization

Questions?

Supervised by Dr. Mohamed Faysel
Beni-Suef University — 2025/2026

Source Code QR

Live Demo QR